DBA5113 Online Marketplace

Assignment 2

Name	Student ID
TANYA TOOLEY	A0318087X
WANG YING	A0318271H

National University of Singapore

Submission Date: March 6, 2026

Link to code repository

Exercise 1: Epsilon-Greedy Multi-Armed Bandit

Note: The complete Python implementation for Exercise 1 is available in Appendix A.

We consider a 3-arm Bernoulli bandit problem with true reward probabilities $p_1 = 0.3$, $p_2 = 0.4$, $p_3 = 0.5$. The best arm is Arm 3 with the highest expected reward of 0.5.

The ε-greedy algorithm works as follows: at each time step t, with probability ε we explore (select a random arm), and with probability $1 - \varepsilon$ we exploit (select the arm with the highest estimated reward $\hat{\mu}_a$). After pulling arm a_t and observing reward $r_{a_t,t}$, we update the estimate using an incremental mean:

$$\hat{\mu}_{a_t} \leftarrow \frac{\hat{\mu}_{a_t} \times N_{a_t} + r_{a_t,t}}{N_{a_t} + 1}$$

where N_{a_t} is the number of times arm a_t has been pulled.

(i) Fraction of Times Best Arm is Selected

Following the steps of lecture, we get Figure 1 shows the average percentage of times the best arm (Arm 3) selected in intervals of 100 steps (500 simulations).

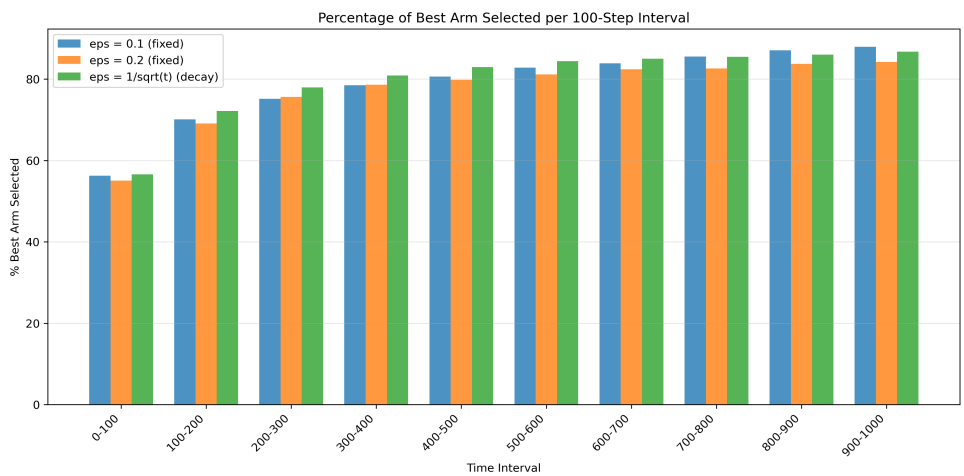

Figure 1: Percentage of best arm selected in intervals of 100 steps

(ii) Different Epsilon Effects

A small ε favors exploitation for faster convergence, while a large ε favors exploration. Decaying ε optimally shifts from early exploration to late exploitation, achieving no-regret learning ($\lim_{T\to\infty} R(T)/T = 0$).

(iii) Effect of More Arms

If there were more than three arms, we would expect the best arm to be selected less frequently and convergence to take longer. This is because with K arms, the theoretical upper bound on

best arm selection is $1 - \varepsilon + \varepsilon/K$ (the probability of exploit is $1 - \epsilon$, then when explore, we still have $1/K$ probability of selecting the best arm), which decreases as K increases, and the algorithm requires more exploration rounds to estimate each arm's reward, delaying accurate identification of the best arm.

(iv) Average Cumulative Regret

The regret at time T measures the cumulative loss from not always selecting the optimal arm:

$$R(T) = \sum_{t=1}^{T} (\mu^* - \mu_{a_t})$$

where $\mu^* = 0.5$ is the best arm's expected reward and μ_{a_t} is the expected reward of the arm selected at time t.

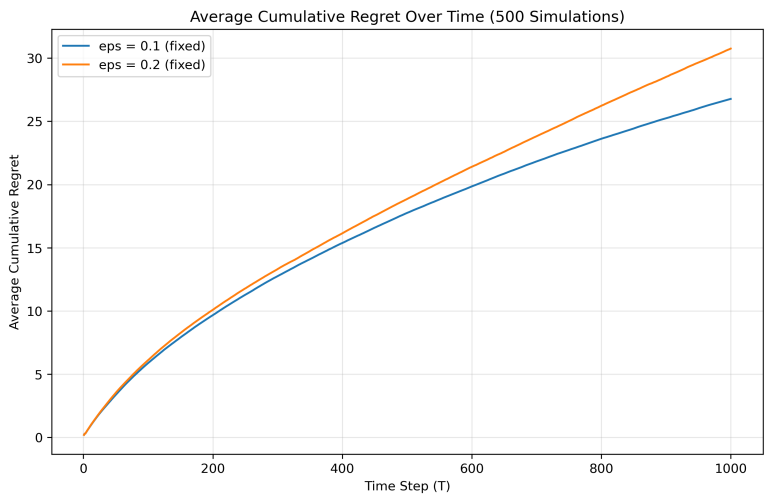

Figure 2: Average cumulative regret over time for $\varepsilon = 0.1$ and $\varepsilon = 0.2$

The regret grows linearly in T for both values of ε. This is expected because ε-greedy explores with a fixed probability ε at every time step, leading to a constant per-step expected regret of $\varepsilon \cdot \bar{\Delta}$, where $\bar{\Delta}$ is the average sub-optimality gap. Higher ε (0.2) leads to higher regret because the algorithm explores more often, incurring more sub-optimal selections (when the underlying reward doesn't change). After $T = 1000$: $\varepsilon = 0.1$ yields average regret ≈ 27.0, while $\varepsilon = 0.2$ yields ≈ 31.5.

Conclusion Exercise 1: It is worth mentioning that while the ε-greedy algorithm is simple and computationally efficient, the choice of ε represents a direct trade-off between adaptability (flexibility) and efficiency (time to converge). A small ε converges quickly in a static environment but struggles to adapt if the underlying reward probabilities change over time. Conversely, a larger ε constantly monitors alternative arms, allowing it to "catch up" faster if the environment shifts, but at the cost of higher steady-state regret.

Exercise 2: Exp3 Algorithm

The probability equation:

$$P_i(t) = (1-\gamma)\frac{W_i(t)}{\sum_j W_j(t)} + \frac{\gamma}{|A|}$$

(i) How Exp3 Balances Exploration and Exploitation

Exp3 balances exploration and exploitation by mixing a weighted probability distribution via Weights $(1-\gamma)$ and a uniform distribution via Exploration (γ), while using importance-weighting to ensure rarely-chosen arms still receive fair credit.

(ii) Exploration Comparison: Exp3 vs Epsilon-Greedy

When $\gamma = \varepsilon$, Exp3 explores more than Epsilon-Greedy. Epsilon-Greedy 'exploits' with $(1-\varepsilon)$ probability to choose only the single best arm 100 percent of the time, whereas Exp3 'exploits' with $(1-\gamma)$ probability using a weighted distribution across all arms. By distributing its exploitation on suboptimal arms, Exp3 inherently maintains a higher level of exploration.

(iii) Area of application of Exp3

Exp3 is best deployed in dynamic reward environments where the underlying reward distributions change over time (e.g., algorithmic pricing against competitors, or shifting user trends). While Epsilon-Greedy relies on averages reward that are slow to adapt to sudden changes, Exp3 maintains a continuous, weighted exploration of all arms and updates its probabilities exponentially. This exponential scaling allows it to adapt to shifting environments dramatically faster.

Exercise 3: Optimal Fee Design

Note: The complete Python implementation for Exercise 3 is available in Appendix B.

(i) Two Approaches to Choose b

Approach 1: A/B Testing (More like Switchback Experiments)

Run controlled experiments where buyer fee splits b are tested on all users, but we apply them sequentially and randomly on daily/weekly basis, and we put also no treatment period between phases to remove the carryover effects. measuring the resulting platform revenue.

Advantage: Directly measures causal effect of b on revenue without needing to model buyer and seller behavior (model-free), giving reliable real-world estimates.

Disadvantage: Requires exposing users to potentially suboptimal fee splits during the experiment, which incurs opportunity cost and may take significant time to reach statistical significance across all candidate values of b.

Approach 2: Multi-Armed Bandit (MAB)

Treat each candidate value of b as an arm and use a bandit algorithm (e.g., Thompson Sampling or UCB) to dynamically learn the best fee split while minimizing regret during the learning process.

Advantage: Learns the optimal b while minimizing cumulative revenue loss during exploration (unlike A/B testing which equally allocates traffic to all arms), adapting quickly as data accumulates.

Disadvantage: Standard MAB assumes stationary rewards, but changing b affects seller pricing strategies (sellers use adaptive algorithms like Exp3), violating the stationarity assumption and potentially leading the MAB to converge to a suboptimal b.

(ii) Simulation Results

The simulation result is shown in Figure 3 and Table 1.

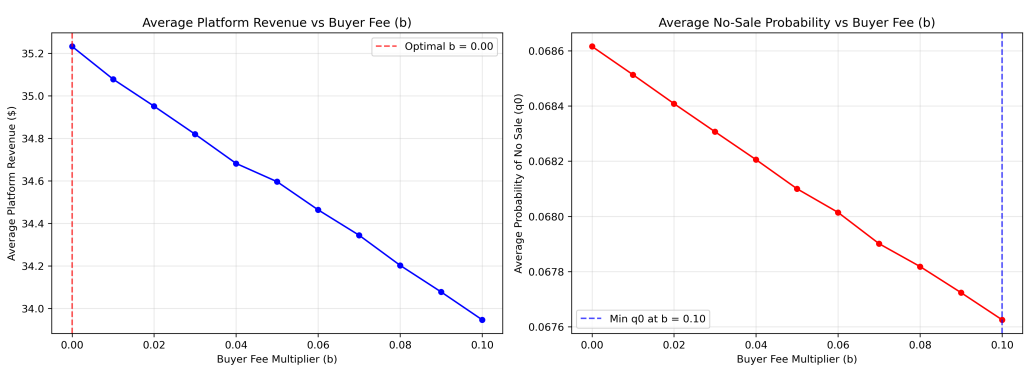

Figure 3: Average platform revenue and probability of no sale (q_0) as a function of b

Buyer Fee (b)	Avg Revenue ($)	Avg q_0
0.00	35.23	0.0686
0.01	35.08	0.0685
0.02	34.95	0.0684
0.03	34.82	0.0683
0.04	34.68	0.0682
0.05	34.60	0.0681
0.06	34.46	0.0680
0.07	34.34	0.0679
0.08	34.20	0.0678
0.09	34.08	0.0677
0.10	33.95	0.0676

Table 1: Simulation results for optimal fee design

The value of b that maximizes platform revenue is $b^* = 0.00$ ($35.23), and the value that minimizes q_0 is $b_{\min} = 0.10$ ($q_0 = 0.0676$).

Interestingly, as the fee burden shifts toward buyers (higher b), platform revenue drops, but market activity actually increases (the probability of no sale q_0 drops to 0.0676 at $b = 0.10$). This occurs because the Exp3-driven sellers aggressively lower their base prices to compensate for the higher buyer fees. Since the platform's revenue is a fixed 10% cut of the base price, the sellers' price reductions cause the platform's total revenue to drop, despite the slight increase in sales volume.

(iii) Importance of Minimizing q_0

Minimizing the probability of no sale (q_0) is crucial because losing buyers to "no sale" events damages long-term platform health by lowering buyer retention, weakening network effects for sellers, and eroding user trust.

(iv) Economic Intuition for Maximizing Revenues

The economic intuition for maximizing platform revenue is to set $b = 0.00$ (placing all fees on sellers), because the platform takes a fixed 10% cut of the seller's base price P. Since buyers make purchasing decisions based on the total effective price $P(1+b)$, shifting the fee b to buyers forces sellers to lower their base price P to attract buyers, which mathematically shrinks the platform's revenue cut.

(v) Disadvantages of Standard MAB for Learning b

1. **Non-stationarity:** Sellers will adapt their pricing strategies (unknown pattern) to react to changes in b, so the reward distribution for each arm (each value of b) is non-stationary, violating the MAB assumption that each arm has a fixed reward distribution.
2. **Competitive Settings:** Algorithms like UCB or TS perform poorly when rewards depend on the actions of strategic competitors, when the platform changes b, the competing sellers

react by adjusting their pricing strategies, when we don't know their strategies, it will be hard to predict the dynamics and to reliably learn the optimal b.

(vi) Switchback Experiment Design

To choose the two fees to test, we should select $b = 0.00$ and $b = 0.10$ which ensures the values are sufficiently different to detect a meaningful statistical effect. The key limitation is that seller pricing algorithms (Exp3) carry over state between periods, creating **carryover effects** that bias the treatment effect estimate.

Exercise 4: DoorDash Messaging Optimization

Note: The complete Python implementation for Exercise 4 is available in Appendix C.

(i) DoorDash's Choice of Thompson Sampling

DoorDash chose MAB with TS over other bandit algorithms because it naturally handles the exploration-exploitation trade-off through posterior sampling, which is computationally efficient for large-scale systems, and it performs well empirically with binary outcomes (accept/reject).

They preferred it over offline ML methods because TS can **learn and adapt in real-time** without a separate training phase, handles the cold-start problem for new Dashers through informative priors, and avoids the feedback loop issues of offline models that are trained on non-representative historical data.

They tested the algorithm using **switchback experiments** (alternating between TS and random/baseline messaging in different time periods) to measure the causal impact on Dasher conversion rates and supply availability.

(ii) Switchback Experiment Simulation

The simulation result shows that Thompson Sampling achieves a significantly higher conversion rate (72.50%) compared to random messaging (49.78%), with a highly significant ATE of 0.2272. Figure 4 and Table 2 summarize the simulation results.

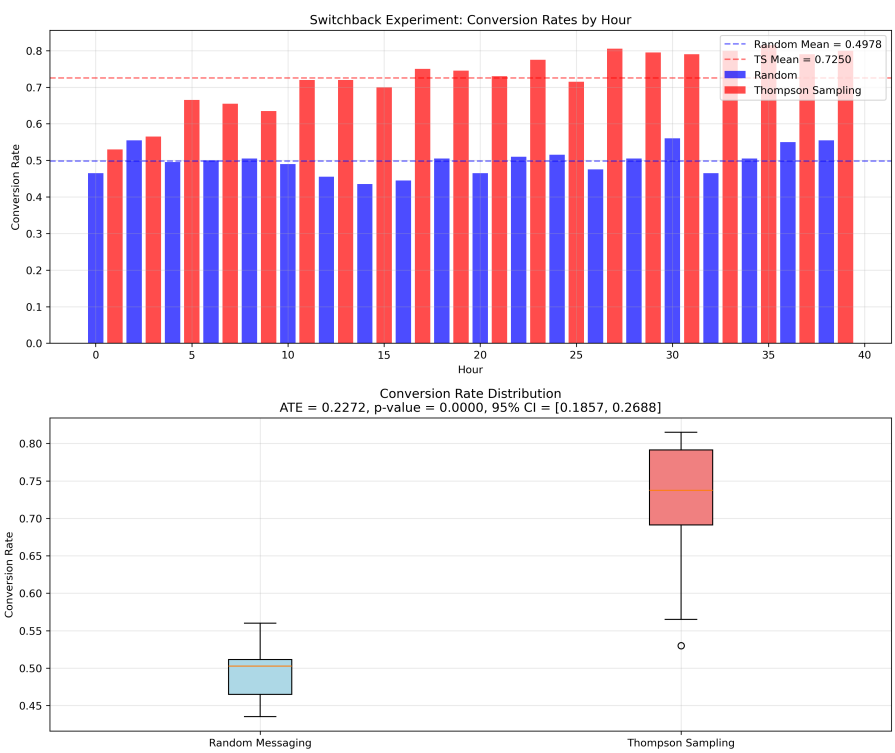

Figure 4: Switchback experiment results: hourly conversion rates and distribution comparison

Metric	Value
Mean Random Conversion Rate	0.4978
Mean TS Conversion Rate	0.7250
Average Treatment Effect (ATE)	0.2272
p-value	< 0.0001
95% Confidence Interval	[0.1857, 0.2688]

Table 2: Switchback experiment analysis results

(iii) Why the ATE is Misleading

The ATE from this switchback experiment is misleading because it **overestimates** the true treatment effect due to a violation of SUTVA. Specifically, the experiment suffers from strong **carryover effects** (or spillover). Because the same population of Dashers is used across alternating hours, messaging a Dasher in one hour directly impacts their availability and likelihood to accept a delivery in the next hour. Therefore, the outcomes observed during the control (Random) periods are heavily contaminated by the actions taken during the preceding treatment (Thompson Sampling) periods, violating the assumption of no interference.

(iv) Possible Solution

To mitigate these carryover effects, we should introduce washout periods between the control and treatment phases. By adding a sufficient time buffer where we exclude data from our analysis immediately after an algorithm switch, we allow the Dashers' states to reset.

A Exercise 1 Complete Code

Below is the complete Python code for Exercise 1, which implements the ε-greedy multi-armed bandit algorithm.

Listing 1: exercise1.py – Complete Implementation

```python
import numpy as np
import matplotlib.pyplot as plt
import os


def epsilon_greedy(T, epsilon_schedule, true_probs, rng):
    """
    Run epsilon-greedy algorithm for T steps.

    Args:
        T: number of time steps
        epsilon_schedule: either a float (fixed) or 'decay' for 1/sqrt(t)
        true_probs: list of true Bernoulli reward probabilities for each arm
        rng: numpy random generator

    Returns:
        actions: array of chosen actions at each step
        rewards: array of observed rewards at each step
    """
    K = len(true_probs)
    # Estimated reward for each arm
    Q = np.zeros(K)
    # Number of times each arm has been pulled
    N_pulls = np.zeros(K)

    actions = np.zeros(T, dtype=int)
    rewards = np.zeros(T)

    # We pull the arm at least once to initialize the expected reward and N_pulls
    for a in range(K):
        # We assume T >= K for this to make sense
        if a < T:
            action = a
            reward = 1.0 if rng.random() < true_probs[action] else 0.0

            N_pulls[action] = 1
            Q[action] = reward

            actions[a] = action
```

```python
            rewards[a] = reward

    # We loop through T pulls, update expected reward and N_pulls
    for t in range(K, T):
        # Determine epsilon for this step (note: t is 0-indexed, so t+1 is the true time step)
        if epsilon_schedule == "decay":
            eps = 1.0 / np.sqrt(t + 1)
        else:
            eps = epsilon_schedule

        # Epsilon-greedy action selection
        if rng.random() < eps:
            action = rng.integers(0, K)
        else:
            max_q = np.max(Q)
            best_arms = np.where(Q == max_q)[0]
            action = rng.choice(best_arms)

        # Play and observe
        reward = 1.0 if rng.random() < true_probs[action] else 0.0

        # Update estimates (using exact formula from lecture slide)
        #    at    = (  at    * Nat + rat,t) / (Nat + 1)
        Q[action] = (Q[action] * N_pulls[action] + reward) / (N_pulls[action] + 1)
        N_pulls[action] += 1

        actions[t] = action
        rewards[t] = reward

    return actions, rewards

def run_simulations(T, epsilon_schedule, true_probs, n_sims=500, seed=42):
    """
    Run multiple simulations of epsilon-greedy.

    Returns:
        best_arm_fraction: fraction of times best arm is selected at each t
        avg_regret: average cumulative regret at each t
    """
    best_arm = np.argmax(true_probs)
    best_prob = true_probs[best_arm]

    best_arm_counts = np.zeros(T)
    cumulative_regrets = np.zeros(T)

```

```python
    for sim in range(n_sims):
        rng = np.random.default_rng(seed + sim)
        actions, rewards = epsilon_greedy(T, epsilon_schedule, true_probs, rng)

        # Track best arm selection
        best_arm_selected = (actions == best_arm).astype(float)
        best_arm_counts += best_arm_selected

        # Compute cumulative regret: sum of (best_prob - prob of chosen arm)
        per_step_regret = np.array([best_prob - true_probs[a] for a in actions])
        cumulative_regrets += np.cumsum(per_step_regret)

    best_arm_fraction = best_arm_counts / n_sims
    avg_regret = cumulative_regrets / n_sims

    return best_arm_fraction, avg_regret


def compute_interval_percentages(best_arm_fraction, interval=100):
    """
    Compute average percentage of best arm selection in intervals.
    E.g., intervals of 100: [0-100), [100-200), ..., [900-1000)
    """
    T = len(best_arm_fraction)
    n_intervals = T // interval
    percentages = []

    for i in range(n_intervals):
        start = i * interval
        end = (i + 1) * interval
        pct = best_arm_fraction[start:end].mean() * 100
        percentages.append(pct)

    return percentages


def plot_best_arm_intervals(results, T, interval=100, display=False):
    """Plot percentage of best arm selected in intervals of 100 steps.
    """
    os.makedirs("figures", exist_ok=True)

    n_intervals = T // interval
    x_labels = [f"{i*interval}-{(i+1)*interval}" for i in range(n_intervals)]
    x_pos = np.arange(n_intervals)
    bar_width = 0.25
```

```python

    plt.figure(figsize=(12, 6))

    for idx, (label, (fraction, _)) in enumerate(results.items()):
        percentages = compute_interval_percentages(fraction, interval)
        plt.bar(x_pos + idx * bar_width, percentages, bar_width,
                label=label, alpha=0.8)

    plt.xlabel("Time Interval")
    plt.ylabel("% Best Arm Selected")
    plt.title("Percentage of Best Arm Selected per 100-Step Interval")
    plt.xticks(x_pos + bar_width, x_labels, rotation=45, ha="right")
    plt.legend()
    plt.grid(True, alpha=0.3, axis="y")
    plt.tight_layout()

    if display:
        plt.show()
    else:
        plt.savefig("figures/best_arm_fraction.png", dpi=300,
            bbox_inches="tight")
        plt.close()


def plot_regret(results_subset, T, display=False):
    """Plot average cumulative regret over time for fixed epsilon only
        ."""
    os.makedirs("figures", exist_ok=True)

    plt.figure(figsize=(10, 6))
    for label, (_, regret) in results_subset.items():
        plt.plot(range(1, T + 1), regret, label=label)

    plt.xlabel("Time Step (T)")
    plt.ylabel("Average Cumulative Regret")
    plt.title("Average Cumulative Regret Over Time (500 Simulations)")
    plt.legend()
    plt.grid(True, alpha=0.3)

    if display:
        plt.show()
    else:
        plt.savefig("figures/regret_comparison.png", dpi=300,
            bbox_inches="tight")
        plt.close()


def main():
    # Problem parameters
```

```python
    true_probs = [0.3, 0.4, 0.5]  # Bernoulli reward probabilities
    T = 1000
    n_sims = 500

    # Three epsilon schedules
    schedules = {
        "eps = 0.1 (fixed)": 0.1,
        "eps = 0.2 (fixed)": 0.2,
        "eps = 1/sqrt(t) (decay)": "decay",
    }

    results = {}
    for label, schedule in schedules.items():
        print(f"Running {label}...")
        fraction, regret = run_simulations(T, schedule, true_probs, n_sims)
        results[label] = (fraction, regret)

        # Report summary per 100-step intervals
        percentages = compute_interval_percentages(fraction)
        print(f"  Best arm % per interval: {[f'{p:.1f}%' for p in percentages]}")
        print(f"  Final cumulative regret: {regret[-1]:.2f}")

    # Plot 1: Best arm fraction in intervals (all 3 schedules)
    plot_best_arm_intervals(results, T)

    # Plot 2: Regret (only fixed epsilon = 0.1 and 0.2)
    regret_subset = {k: v for k, v in results.items() if "fixed" in k}
    plot_regret(regret_subset, T)

    print("\nFigures saved to figures/")


if __name__ == "__main__":
    main()
```

B Exercise 3 Complete Code

Below is the complete Python code for Exercise 3, which simulates the optimal fee design with Exp3 sellers and multinomial logit demand.

Listing 2: exercise3.py – Complete Implementation

```python
import numpy as np
import matplotlib.pyplot as plt
import os

```

```python

# -- Retailer parameters (from Table 1) ----------------------------------
ALPHA = np.array([0.62, 0.7, 0.75])        # Base attractiveness
BETA  = np.array([3.32, 3.1, 3.55])        # Own price sensitivity
GAMMA_CROSS = np.array([1.5, 1.1, 1.22])   # Competitor price sensitivity
S     = np.array([4.6, 4.7, 4.5])          # Quality score
DELTA = np.array([0.15, 0.15, 0.2])        # Quality sensitivity
B_IND = np.array([0, 0, 1])                # Badge indicator
THETA = np.array([0.3, 0.3, 0.3])          # Badge effect

# Price options for retailers
PRICE_OPTIONS = np.array([0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0])


def compute_purchase_probabilities(prices, b):
    """
    Compute purchase probabilities using multinomial logit model.

    Args:
        prices: array of shape (3,) -- prices set by each retailer
        b: buyer fee multiplier

    Returns:
        probs: array of shape (4,) -- [q0, q1, q2, q3]
    """
    V = np.zeros(3)
    effective_prices = prices * (1 + b)

    for i in range(3):
        cross_price_sum = sum(prices[j] * (1 + b) for j in range(3) if j != i)
        V[i] = (ALPHA[i]
                - BETA[i] * effective_prices[i]
                + GAMMA_CROSS[i] * cross_price_sum
                + DELTA[i] * S[i]
                + THETA[i] * B_IND[i])

    # V0 = 0 (outside option)
    exp_V = np.exp(V)
    denom = 1.0 + np.sum(exp_V)  # 1 for exp(V0)=exp(0)=1

    q0 = 1.0 / denom
    q = exp_V / denom

    return np.concatenate(([q0], q))

```

```python

class Exp3Seller:
    """Exp3 algorithm for a single seller."""

    def __init__(self, n_actions, gamma, rng):
        self.n_actions = n_actions
        self.gamma = gamma
        self.rng = rng
        self.weights = np.ones(n_actions)

    def get_probabilities(self):
        """Compute action probabilities."""
        total_weight = np.sum(self.weights)
        probs = (1 - self.gamma) * (self.weights / total_weight) + self.gamma / self.n_actions
        return probs

    def choose_action(self):
        """Choose an action according to Exp3 probabilities."""
        probs = self.get_probabilities()
        action = self.rng.choice(self.n_actions, p=probs)
        return action

    def update(self, action, reward):
        """Update weights based on observed reward."""
        probs = self.get_probabilities()
        # Estimated reward (importance-weighted)
        estimated_reward = reward / probs[action]
        # Update weight for chosen action
        self.weights[action] *= np.exp(self.gamma * estimated_reward / self.n_actions)


def simulate_one_run(b, T, gamma_exp3, rng):
    """
    Run one simulation of T time steps.

    Returns:
        total_platform_revenue: total platform revenue over T steps
        total_q0: sum of q0 over T steps (for averaging later)
    """
    n_actions = len(PRICE_OPTIONS)
    sellers = [Exp3Seller(n_actions, gamma_exp3, rng) for _ in range(3)]

    total_platform_revenue = 0.0
    total_q0 = 0.0

    for t in range(T):
```

```python
        # Each seller chooses a price action
        actions = [seller.choose_action() for seller in sellers]
        prices = PRICE_OPTIONS[actions]

        # Compute purchase probabilities
        probs = compute_purchase_probabilities(prices, b)
        q0 = probs[0]
        total_q0 += q0

        # Buyer makes purchase decision
        buyer_choice = rng.choice(4, p=probs)  # 0=no buy, 1-3=retailer

        if buyer_choice == 0:
            # Update sellers with 0 reward for their chosen actions
            for i, seller in enumerate(sellers):
                seller.update(actions[i], 0.0)
        else:
            # Sale to retailer (buyer_choice - 1)
            chosen_retailer = buyer_choice - 1
            chosen_price = prices[chosen_retailer]

            # Platform revenue = total fee = 10% of transaction price
            # The total fee is always 10%: buyer pays P*(1+b), seller receives P*(0.9+b)
            # Platform revenue = P*(1+b) - P*(0.9+b) = P*0.1
            platform_revenue = chosen_price * 0.10
            total_platform_revenue += platform_revenue

            # Seller revenue = price * (0.9 + b) -- what they receive
            # But for Exp3, we normalize reward to [0,1]
            for i, seller in enumerate(sellers):
                if i == chosen_retailer:
                    # Seller's reward: normalized revenue
                    # Seller receives P*(0.9+b), normalize by max possible: 1.0*(0.9+0.10)=1.0
                    seller_revenue = chosen_price * (0.9 + b)
                    reward = min(seller_revenue, 1.0)  # Cap at 1
                    seller.update(actions[i], reward)
                else:
                    seller.update(actions[i], 0.0)

    return total_platform_revenue, total_q0 / T


def run_all_simulations(b_values, T=1000, n_runs=200, gamma_exp3=0.2, base_seed=42):
    """
    Run simulations for all values of b.
```

```python

    Returns:
        avg_revenues: dict b -> average platform revenue
        avg_q0s: dict b -> average q0
    """
    avg_revenues = {}
    avg_q0s = {}

    for b in b_values:
        revenues = []
        q0s = []

        for run in range(n_runs):
            rng = np.random.default_rng(base_seed + run)
            rev, q0 = simulate_one_run(b, T, gamma_exp3, rng)
            revenues.append(rev)
            q0s.append(q0)

        avg_revenues[b] = np.mean(revenues)
        avg_q0s[b] = np.mean(q0s)

        print(f"  b = {b:.2f}: avg revenue = {avg_revenues[b]:.2f},
           avg q0 = {avg_q0s[b]:.4f}")

    return avg_revenues, avg_q0s


def plot_results(b_values, avg_revenues, avg_q0s, display=False):
    """Plot average platform revenue and q0 vs b."""
    os.makedirs("figures", exist_ok=True)

    rev_list = [avg_revenues[b] for b in b_values]
    q0_list = [avg_q0s[b] for b in b_values]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    # Revenue plot
    ax1.plot(b_values, rev_list, 'b-o', markersize=5)
    best_b_rev = b_values[np.argmax(rev_list)]
    ax1.axvline(x=best_b_rev, color='r', linestyle='--', alpha=0.7,
                label=f'Optimal b = {best_b_rev:.2f}')
    ax1.set_xlabel('Buyer Fee Multiplier (b)')
    ax1.set_ylabel('Average Platform Revenue ($)')
    ax1.set_title('Average Platform Revenue vs Buyer Fee (b)')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # q0 plot
    ax2.plot(b_values, q0_list, 'r-o', markersize=5)
```

```python
    best_b_q0 = b_values[np.argmin(q0_list)]
    ax2.axvline(x=best_b_q0, color='b', linestyle='--', alpha=0.7,
                label=f'Min q0 at b = {best_b_q0:.2f}')
    ax2.set_xlabel('Buyer Fee Multiplier (b)')
    ax2.set_ylabel('Average Probability of No Sale (q0)')
    ax2.set_title('Average No-Sale Probability vs Buyer Fee (b)')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()

    if display:
        plt.show()
    else:
        plt.savefig('figures/fee_optimization.png', dpi=300,
            bbox_inches='tight')
        plt.close()


def main():
    b_values = np.round(np.arange(0.00, 0.11, 0.01), 2)

    print(f"\nSimulating for b in {{{', '.join(f'{b:.2f}' for b in
        b_values)}}}")
    print(f"T = 1000, Runs = 200, gamma = 0.2\n")

    avg_revenues, avg_q0s = run_all_simulations(b_values, T=1000,
        n_runs=200)

    rev_list = [avg_revenues[b] for b in b_values]
    q0_list = [avg_q0s[b] for b in b_values]

    best_b_rev = b_values[np.argmax(rev_list)]
    best_b_q0 = b_values[np.argmin(q0_list)]

    print(f"Optimal b for max revenue:  {best_b_rev:.2f} (revenue = ${
        max(rev_list):.2f}), Optimal b for min q0: {best_b_q0:.2f} (q0
        = {min(q0_list):.4f})")

    plot_results(b_values, avg_revenues, avg_q0s)


if __name__ == "__main__":
    main()
```

C Exercise 4 Complete Code

Below is the complete Python code for Exercise 4, which implements Thompson Sampling messaging and the switchback experiment.

Listing 3: exercise4.py – Complete Implementation

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import os


def generate_dashers(n_dashers=1000, seed=42):
    """
    Generate dasher features and true acceptance probabilities.

    Each dasher has features Xi = (xi1, xi2, xi3) ~ N(0,1)
    Acceptance probability: Pi = 1 / (1 + exp(-(b1*xi1 + b2*xi2 + b3*
        xi3)))
    b1 = 1.2, b2 = -0.8, b3 = 0.5
    """
    rng = np.random.default_rng(seed)

    # Feature matrix: n_dashers x 3
    X = rng.standard_normal((n_dashers, 3))

    # Coefficients
    beta = np.array([1.2, -0.8, 0.5])

    # Logit probabilities
    logits = X @ beta
    true_probs = 1.0 / (1.0 + np.exp(-logits))

    return X, true_probs


def random_messaging(true_probs, n_select=200, rng=None):
    """
    Random messaging: select 200 dashers at random.
    Returns selected indices and their binary responses.
    """
    n_dashers = len(true_probs)
    selected = rng.choice(n_dashers, size=n_select, replace=False)
    responses = rng.random(n_select) < true_probs[selected]
    return selected, responses.astype(int)


def thompson_sampling(alpha_params, beta_params, true_probs, n_select
```

```
=200, rng=None):
    """
    Thompson Sampling: sample from Beta posteriors, select top-200.
    Returns selected indices and their binary responses.
    """
    n_dashers = len(true_probs)

    # Sample from Beta(alpha, beta) for each dasher
    sampled_probs = rng.beta(alpha_params, beta_params)

    # Select top-200 dashers with highest sampled probabilities
    selected = np.argsort(sampled_probs)[-n_select:]

    # Generate actual responses based on true probabilities
    responses = rng.random(n_select) < true_probs[selected]

    return selected, responses.astype(int)


def run_switchback_experiment(true_probs, n_hours=40, n_select=200, seed=42):
    """
    Run switchback experiment alternating Random and TS every hour.

    Returns:
        hourly_data: list of dicts with hour info
    """
    rng = np.random.default_rng(seed)
    n_dashers = len(true_probs)

    # Thompson Sampling priors: Beta(1, 1) for each dasher
    alpha_params = np.ones(n_dashers)
    beta_params = np.ones(n_dashers)

    hourly_data = []

    for hour in range(n_hours):
        # Alternate: even hours = Random, odd hours = Thompson Sampling
        if hour % 2 == 0:
            method = "Random"
            selected, responses = random_messaging(true_probs, n_select, rng)
        else:
            method = "Thompson Sampling"
            selected, responses = thompson_sampling(
                alpha_params, beta_params, true_probs, n_select, rng
            )

```

```python
        # Update Thompson Sampling posteriors (always update, even during random hours)
        # This simulates that DoorDash learns from ALL interactions
        alpha_params[selected] += responses
        beta_params[selected] += (1 - responses)

        conversion_rate = responses.mean()
        hourly_data.append({
            "hour": hour,
            "method": method,
            "selected": selected,
            "responses": responses,
            "conversion_rate": conversion_rate,
            "n_conversions": responses.sum(),
        })

    return hourly_data


def analyze_switchback(hourly_data):
    """
    Analyze switchback experiment results.
    Returns ATE, p-value, 95% CI.
    """
    random_rates = [d["conversion_rate"] for d in hourly_data if d["method"] == "Random"]
    ts_rates = [d["conversion_rate"] for d in hourly_data if d["method"] == "Thompson Sampling"]

    random_rates = np.array(random_rates)
    ts_rates = np.array(ts_rates)

    # Average Treatment Effect
    ate = np.mean(ts_rates) - np.mean(random_rates)

    # Two-sample t-test
    t_stat, p_value = stats.ttest_ind(ts_rates, random_rates)

    # 95% confidence interval for the difference in means
    n1, n2 = len(ts_rates), len(random_rates)
    se = np.sqrt(np.var(ts_rates, ddof=1) / n1 + np.var(random_rates, ddof=1) / n2)
    t_crit = stats.t.ppf(0.975, df=min(n1, n2) - 1)
    ci_lower = ate - t_crit * se
    ci_upper = ate + t_crit * se

    return {
        "ate": ate,
        "p_value": p_value,
```

```
        "ci_lower": ci_lower,
        "ci_upper": ci_upper,
        "mean_random": np.mean(random_rates),
        "mean_ts": np.mean(ts_rates),
        "random_rates": random_rates,
        "ts_rates": ts_rates,
    }


def plot_results(hourly_data, analysis, display=False):
    """Plot switchback experiment results."""
    os.makedirs("figures", exist_ok=True)

    # Plot 1: Conversion rates over time
    fig, axes = plt.subplots(2, 1, figsize=(12, 10))

    hours = [d["hour"] for d in hourly_data]
    rates = [d["conversion_rate"] for d in hourly_data]
    colors = ['blue' if d["method"] == "Random" else 'red' for d in hourly_data]

    ax1 = axes[0]
    for h, r, c, d in zip(hours, rates, colors, hourly_data):
        ax1.bar(h, r, color=c, alpha=0.7,
                label=d["method"] if h <= 1 else "")
    ax1.set_xlabel('Hour')
    ax1.set_ylabel('Conversion Rate')
    ax1.set_title('Switchback Experiment: Conversion Rates by Hour')
    ax1.axhline(y=analysis["mean_random"], color='blue', linestyle='--',
                alpha=0.5, label=f'Random Mean = {analysis["mean_random"]:.4f}')
    ax1.axhline(y=analysis["mean_ts"], color='red', linestyle='--',
                alpha=0.5, label=f'TS Mean = {analysis["mean_ts"]:.4f}')
    ax1.legend(loc='upper right')
    ax1.grid(True, alpha=0.3)

    # Plot 2: Comparison box plot
    ax2 = axes[1]
    bp = ax2.boxplot([analysis["random_rates"], analysis["ts_rates"]],
                     tick_labels=["Random Messaging", "Thompson Sampling"],
                     patch_artist=True)
    bp['boxes'][0].set_facecolor('lightblue')
    bp['boxes'][1].set_facecolor('lightcoral')
    ax2.set_ylabel('Conversion Rate')
    ax2.set_title(f'Conversion Rate Distribution\n'
                  f'ATE = {analysis["ate"]:.4f}, '
```

```python
                    f'p-value = {analysis["p_value"]:.4f}, '
                    f'95% CI = [{analysis["ci_lower"]:.4f}, {analysis["ci_upper"]:.4f}]')
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()

    if display:
        plt.show()
    else:
        plt.savefig('figures/switchback_experiment.png', dpi=300, bbox_inches='tight')
        plt.close()


def main():
    print("\nGenerating 1000 dashers...")
    X, true_probs = generate_dashers(n_dashers=1000, seed=42)
    print(f"  True acceptance probability range: [{true_probs.min():.4f}, {true_probs.max():.4f}]")
    print(f"  Mean true acceptance probability:   {true_probs.mean():.4f}")

    print("\nRunning switchback experiment (40 hours)...")
    hourly_data = run_switchback_experiment(true_probs, n_hours=40, n_select=200, seed=42)

    print(f"\n{'Hour':>4} {'Method':<20} {'Conv. Rate':>10} {'Conversions':>12}")
    print("-" * 50)
    for d in hourly_data:
        print(f"{d['hour']:>4} {d['method']:<20} {d['conversion_rate']:>10.4f} {d['n_conversions']:>12}")

    analysis = analyze_switchback(hourly_data)

    print(f"  Mean Random Conversion Rate:    {analysis['mean_random']:.4f}, Mean TS Conversion Rate: {analysis['mean_ts']:.4f}")
    print(f"  Average Treatment Effect (ATE): {analysis['ate']:.4f}, p-value: {analysis['p_value']:.4f}, 95% Confidence Interval: [{analysis['ci_lower']:.4f}, {analysis['ci_upper']:.4f}]")

    plot_results(hourly_data, analysis)

if __name__ == "__main__":
    main()
```